Concurrent Data Processing System Using Java and Go

Amjad Alqahtani
Computer Science, University of Cumberlands
MSCS-632: Advanced Programming Languages
Dr. Jay Thom
Oct 11, 2026

Concurrent Data Processing System Using Java and Go

The purpose of this assignment is to demonstrate concurrent multithreading data processing written both in java and go. The assignment uses multiple workers to execute tasks and each work is a separate thread and a shared queue. Shared resources have to have access synchronization, error handling and safe access.

Java Implementation details

On the implementation level, Java has few functions. First, Shared Queue which plays as a shared resource among concurrent threads called TaskQueue. It will use synchronized methods to prevent race conditions. Basically, I am using a linked list data structure, and controlling access to a shared resource so that only one thread at a time can be executed. Technically, Java must acquire a lock before executing the addTask() and getTask() methods.

I have a number of worker threads equals to three worker threads. The threads are created by using the Thread class. Each worker retrieves tasks from a shared queue, processes them, and stores the results in a shared list. The start() method starts the threads so they can run concurrently, while the join() method makes the main program wait until all workers finish. The three worker threads store their processed results in a shared resource called shared list. I use synchronized(results) to ensure that only one thread can access the protected section of code at a time. This prevents race conditions and keeps the results safe.

The Java program uses try-catch blocks to handle errors. Specifically, InterruptedException handles situations where the main thread is interrupted while waiting for workers to finish. IOException handles errors that may occur when writing results to the output file. The Java program uses join() to make the main thread wait until all three worker threads finish processing their tasks. This ensures that all workers complete their execution before the program prints and saves the results.

Go Implementation details

On the implementation level, Go has few functions. First, our shared resource is a channel because this is one of the new design choices in Go called chan int that distributes tasks safely named tasks. Go has a goroutine which is a lightweight thread managed by the Go runtime. Three goroutines process tasks concurrently. It is created by placing the go keyword before a

function call. This allows the function to run concurrently with other goroutines. Goroutines can communicate and exchange data using channels.

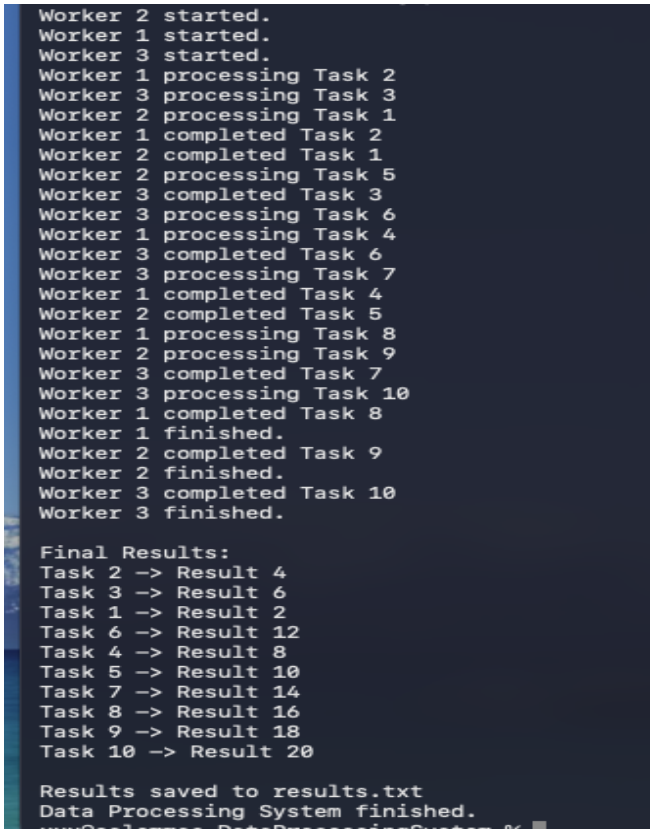
sync.Mutex protects the shared results from concurrent modifications. Since multiple goroutines share the same resource, Go uses a lock to allow only one goroutine at a time to modify it. Other goroutines must wait until the lock is released. This prevents race conditions and ensures safe access to shared data. sync.WaitGroup ensures that the main goroutine waits until all worker goroutines finish their tasks before the program exits. Each worker calls Done() when it finishes, and the main goroutine uses Wait() to wait for all workers to complete.

In Go, functions can return errors when something goes wrong. Go is explicitly checking these errors using if err != nil and handles them to prevent unexpected problems during program execution. Finally, In Go, defer ensures that cleanup operations are performed when a function finishes. It can release resources, such as closing files or unlocking a mutex, and call wg.Done() to mark a worker as finished.

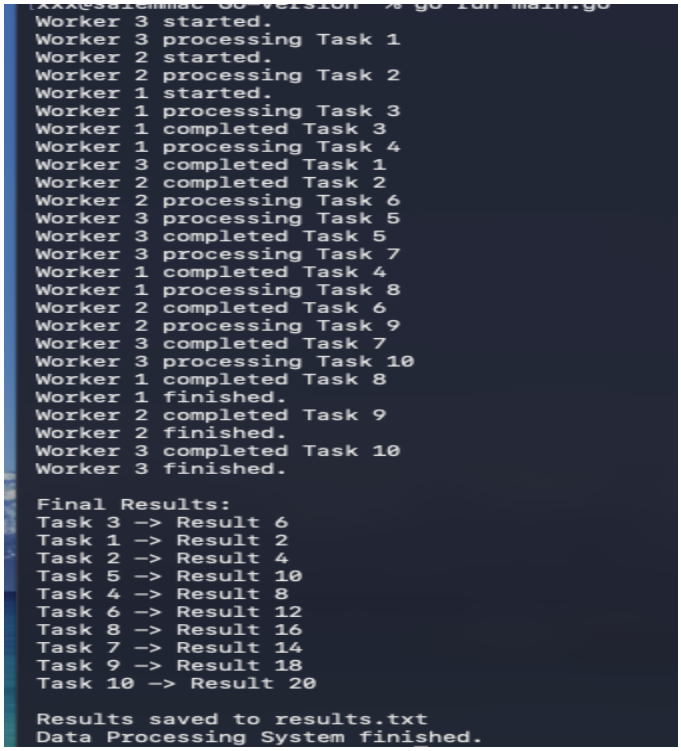
Comparison of Java and Go

Both languages are supporting concurrent data processing, but their approaches are different. Java uses threads to execute concurrent tasks, while Go uses lightweight goroutines. Java uses a shared queue protected by synchronized methods, while Go uses channels to distribute the tasks. Java uses synchronized(results), while Go uses sync.Mutex to allow concurrent writing results. Java uses join() to wait for all threads to finish, while Go uses sync.WaitGroup. For error handling, Java uses try-catch blocks, while Go checks errors explicitly using if err != nil. To clean up, Go uses defer, while Java uses finally and try-with-resources. Finally, Go provides a simpler approach to managing concurrent workers through goroutines and channels.

Screenshot Results : Java



Go



References:

Java Oracle. (n.d.). Lesson: Concurrency. The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/concurrency/

Google. (n.d.). Go by example: Mutexes. https://gobyexample.com/mutexes

Alqahtani, A. (2026). Concurrent_DATA_PROCESSING [Computer software]. GitHub. https://github.com/AmjadAlqahtani90/Concurrent_DATA_PROCESSING